[:digit:]	Digit	*[[:digit:]]?*
[:lower:]	Lower-case alphabet	*[[:lower:]]{5,}*
[:upper:]	Upper-case alphabet	*([[:upper:]]+)?*
[:punct:]	Punctuation	*[[:punct:]]*
[:space:]	All white-space characters including newline, carriage return, and so on.	*[[:space:]]+*

Meta-characters are a type of Perl-style regular expressions that are supported by a subset of text-processing utilities. Not all utilities will support the following notations.

Regex	Description	Example
\b	Word boundary	*\bcool\b* matches only *cool* and not *coolant*.
\B	Non-word boundary	*cool\B* matches *coolant* but not *cool*.
\d	Single digit character	*b\db* matches *b2b* but not *bcb*.
\D	Single non-digit	*b\Db* matches *bcb* but not *b2b*.
\w	Single word character (alnum and _)	*\w matches 1 or a* but not &.
\W	Single non-word character	*\w* matches & but not *1* or *a*.
\n	Newline	*\n* matches a new line.
\s	Single white-space	*x\sx* matches *x x* but not *xx*.
\S	Single non-space	*x\Sx* matches *xkx* but not *x x*.
\r	Carriage return	*\r* matches carriage return.

The above tables can be used as a reference while constructing regular expression patterns.

Let us go through a few examples of regular expressions.

Treatment of special characters

Regular expressions use some characters such as $, ^, ., *, +, {, and } as special characters. But what if we want to use these characters as non-special characters (normal text character)? Let's see an example. regex: [a-z]*.[0-9]

How is this interpreted? It can be zero or more [a-z] ([a-z]*), then any one character (.), and one character in the set [0-9] such that it matches abcdeO9. It can also be interpreted as one of [a-z], then a character *, then a character . (period), and a digit such that it matches x*.8. In order to overcome this problem, we precede the character with a forward slash "\" (doing this is called "escaping the character"). The characters such as * that have multiple meanings are prefixed with "\" to make them into a special meaning or to make them non special. Whether special characters or non-special characters are to be escaped varies depending on the tool that you are using.

In short the term special meaning means that a character is considered as meaningful interpretation other than its character ascii value. For example a* means a, aa, aaa... Here * has special meaning since its not iterpreted as ascii character '*'. Certain characters to be escaped using \ to give special meaning while some others are by default taken as special meaning (Eg. *). To use it as regular ascii meaning, it should be escaped. Here is small list of characters having special meaning with escaping.

Special meaning:

\+, \{, \}, \(, \), \?

Characters that are by default special (You need to escape these inorder to use as regular ascii):

*, ., ^, $, [,]

To match any line containing ONLY the word test, and no other characters on it, use '*^test$*'. This is interpreted as 'start of line marker' followed by '*test*' followed by 'end of line marker'.

Another good example is to extract e-mail addresses from the given text. An e-mail address has the format *username@domain.root*. We can formulate the regular expression as:

[A-Za-z0-9.]+@[A-Za-z0-9.]+\.[a-zA-Z]{2,4}. The *[A-Za-z0-9.]+* before @ states that the given character class should occur one or more times, just as after the @. At the end of the e-mail address, we have the TLD (top-level domain), which can be two to four characters in length, as specified by *{2,4}*.

Points to remember

In Sed, 'one or more' (+) is always prefixed with the backslash escape character if it does not occur after a character set/class, while 'zero or more' (*) is not thus prefixed.

We can do an inverse match using / */PATTERN/!{statements}*. That is, include the bang (!) after the slash after PATTERN.

Sed essentials

Learning Sed involves understanding the basic concepts, and gaining experience with practice. Hence, I will go through different usage contexts and the basic concepts in the following section. Real scripts where Sed can be used will also be explained.

Text replacement with Sed

Sed processes the text input line by line, by default. To perform stream editing on a file, specify the operation, followed by the filename. For example:

```
$ sed 's/PATTERN/REPLACEMENT TEXT/' filename
```

We can also provide the contents of a file to Sed on its standard input, as follows:

```
$ cat filename | sed 's/pattern/replace_string/'
```